

课程笔记：Anti-gravity 2.0 与 Agent Teams

基于 Kevin How (Google) 在 WF2026 的演讲整理

2026 年 7 月



来源：AI Engineer / Kevin How, Google

视频发布日：2026-07-01（会议视频发布日）

本主题时间范围：本主题：04:31:48–04:49:59（约 18 分钟）

原始视频：<https://www.youtube.com/watch?v=htM02KMNZnk&t=16308s>

证据说明：正文以英文原始字幕和经视觉核验的本地视频帧为依据；讲者展示的指标均按演讲陈述处理。

目录

1 从模型能力到产品抽象：为什么要随智能扩展	2
1.1 问题：能力提升为何仍会被产品界面困住	2
1.2 能力演进带来的原语迁移	2
1.3 两次产品迁移留下的边界	3
1.4 Anti-gravity 2.0 的架构判断	3
1.5 本章小结	4
2 Agent Teams：把任务澄清转成动态分工	4
2.1 从并行管理到团队领导	4
2.2 从用户请求到团队执行的协议	4
2.3 为何“更多智能体”不等于“更多可靠性”	5
2.4 规模案例：从应用原型到 OS kernel	6
2.5 本章小结	7
3 研究评测自动化：任务图、事件与交互审阅	7
3.1 一个可迭代的评测问题	7
3.2 从 delta 到可审阅的假设网络	7
3.3 Sidecar 伴随进程 (sidecar)：让系统获得持续的事件入口	8
3.4 生成式界面：让结果按问题形状呈现	9
3.5 本章小结	10
4 总结与延伸：为下一代模型选择可扩展的原语	10
4.1 把三项原语放回同一张产品地图	10
4.2 讲者的收束：产品应当给能力留出接口	11
4.3 给构建者的检查清单	11
4.4 开放问题与谨慎推广	11
4.5 本章小结	11

1 从模型能力到产品抽象：为什么要随智能扩展

1.1 问题：能力提升为何仍会被产品界面困住

Kevin How 用足球作开场：在比赛最后时刻拥有 Messi，战术就是“把球交给 Messi，然后让开”。这段比喻讨论的并非放弃产品设计，而是产品应当为已经具备规划与执行能力的模型创造可施展的工作环境。讲者将 Anti-gravity 描述为面向技术及非技术用户的 agentic coding 产品，并说明本次演讲聚焦其背后的设计原则与路线图。¹

随智能扩展 (scaling with intelligence) 指：服务模型变强时，产品的交互原语与用户体验也应当随之演进，使用户能够实际使用模型已经抵达的能力边界。它是讲者提出的产品设计原则，不能理解为参数量、调用次数或功能数量的单向增加。^a

^a视频原始时间：04:34:14–04:34:35。

这个判断提供了一个产品审视问题：当模型已能完成多步研究、工具调用和任务分解时，产品仍只提供补全框、聊天侧栏或一次性按钮，用户得到的仍可能是局部能力。反过来，功能越多也不自动代表产品更能释放模型能力；关键在于控制、协作、观察和介入的方式是否适合模型当前能承担的工作。

1.2 能力演进带来的原语迁移

讲者用一张时间线回看开发工具的变化。2022 年的 autocomplete 与 chat 主要建立在 embeddings、规则、文件、代码片段和 Tree-sitter AST parsing 之上，因此上下文检索具有较强确定性。2024 年，agent 进入工作流，custom tools、MCP server integrations 与 permission systems 成为新的产品原语。到 2025 年，Agent Manager 让用户并行管理多个智能体，skills、hooks、artifacts 与 agent-first management UI 开始承载协作过程。²

这条时间线是讲者对自身产品历程的归纳，不能替代整个行业的严格分期。它的教学价值在于揭示一条因果链：模型能可靠承担的工作范围改变后，用户需要操作和审阅的对象也会改变。早期的核心对象是上下文与补全；agent 时代加入工具与授权；多智能体阶段则出现任务、角色、产物和协作关系。

¹视频原始时间：04:32:00–04:34:06。

²视频原始时间：04:34:45–04:35:36。

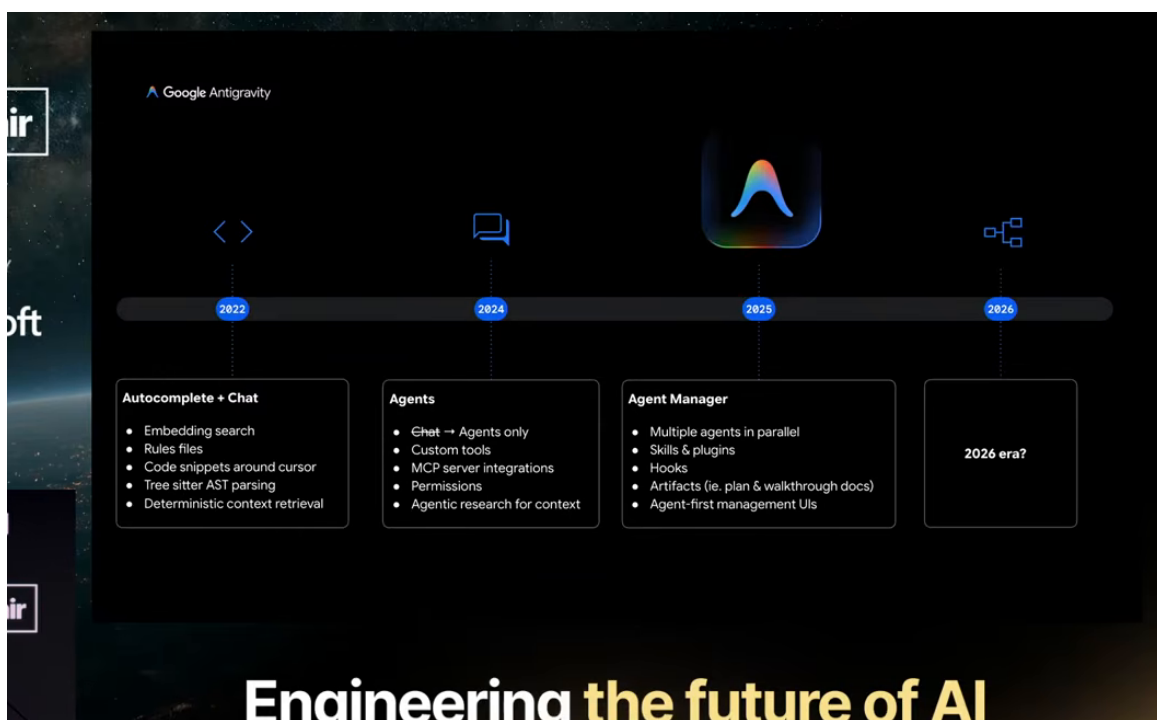


图 1: 讲者的 2022–2026 产品原语演进图。³

可

以把“原语”理解为产品留给用户和模型操作的基本积木。文件、权限、任务、角色、触发器和交互视图都属于不同层面的积木。原语选择决定了系统能否把模型能力转化为可控工作流。

1.3 两次产品迁移留下的边界

讲者没有把迁移描写为无风险过程。第一段经历是“给 AI 一个终端”。用户曾担忧 agent 删除代码库或造成灾难性后果；讲者的结论是，模型能力提高与 permission systems 的投入，使用户能够更快地构建和交付，并让模型更好判断终端中应执行与不应执行的操作。⁴

第二段经历来自 Windsurf 移除 chat sidebar、仅保留 agent 的决定。用户对熟悉交互的反弹很强；讲者回顾后认为，多步 research 与 execution 最终成为更常见的范式。⁵ 这两段经历共同说明：模型能力提升不会自动消除迁移成本，权限、可观测性和用户学习都属于产品迁移的一部分。

更

强的模型并不替代权限系统。终端执行、外部系统连接和多智能体并发会扩大影响范围；权限边界、人工审阅和可追溯证据仍是安全交付的前提。演讲未给出具体授权实现，正文不应虚构其机制。

1.4 Anti-gravity 2.0 的架构判断

Anti-gravity 最初在 IDE 中引入 Agent Manager，之后抽出 agent 并推出独立 CLI；2.0 则将 IDE 与 Agent Manager 解耦为两个应用，使 Agent Manager 可作为独立应用使用。⁶ 讲者提出的层次类

⁴视频原始时间：04:36:03–04:36:32。

⁵视频原始时间：04:36:32–04:37:07。

⁶视频原始时间：04:32:59–04:33:35。

比是：IDE 相对于 Agent Manager，类似 debugger 相对于 IDE。debugger 在需要进入更低抽象层时提供检查入口；同样，IDE 在用户需要深入执行细节时保持价值，Agent Manager 则成为协调团队工作的上层控制面。⁷

代

理编排（agent orchestration）是协调多个智能体的任务、角色、执行与汇总的上层工作方式。讲者将 agent teams、swarms 和 software factories 视为指向同一协作趋势的不同名称；它们都不等同于单个智能体的一次工具调用。^a

^a视频原始时间：04:37:31–04:37:42。

这一解耦表达的是产品边界判断：团队编排成为一类独立工作，IDE 保留为深入代码与执行层的入口。讲者随后给出三类面向 2026 的产品原语；后续章节将依次讨论它们如何回答“谁做工作”“何时被触发”“人如何审阅结果”三个问题。⁸

1.5 本章小结

随智能扩展要求产品同步升级可操作的抽象，不能只在旧界面中堆叠模型功能。讲者的案例表明，终端、权限与 chat sidebar 的迁移都带来用户与安全成本。Anti-gravity 2.0 把 Agent Manager 提升为独立控制面，为后续的团队分工、事件触发与按需交互铺设了架构边界。

2 Agent Teams：把任务澄清转成动态分工

2.1 从并行管理到团队领导

讲者将 Anti-gravity 1.0 的核心概括为：让人处于驾驶位，并行管理多个智能体。产品使用经验又反过来帮助 Gemini 学习如何管理智能体团队；他同时承认，多智能体系统在协作与复杂任务分解上仍有很大提升空间。⁹ 这项限定十分重要：Agent Teams 是以模型能力为前提的产品实验，演讲没有给出已经解决协调质量的证明。

讲者说 Gemini 3.5 Flash 已从执行任务扩展到“leading teams”，并把速度、成本与智能放在同一条权衡曲线上。¹⁰ 模型版本、发布时间和能力定位均属于讲者在演讲中的陈述，教学笔记据此解释机制，不把它们扩展成独立基准结论。

2.2 从用户请求到团队执行的协议

Agent Teams 在 Anti-gravity 中以 public preview 形式出现，讲者展示了‘/team’命令和 teamwork-preview 模式。用户先描述任务，任务越具体越有利；信息不足时，系统可以继续追问，直至工作目标清晰。¹¹

⁷视频原始时间：04:37:09–04:37:33。

⁸视频原始时间：04:37:42–04:38:20。

⁹视频原始时间：04:38:26–04:39:05。

¹⁰视频原始时间：04:39:05–04:39:31。

¹¹视频原始时间：04:39:30–04:40:03。

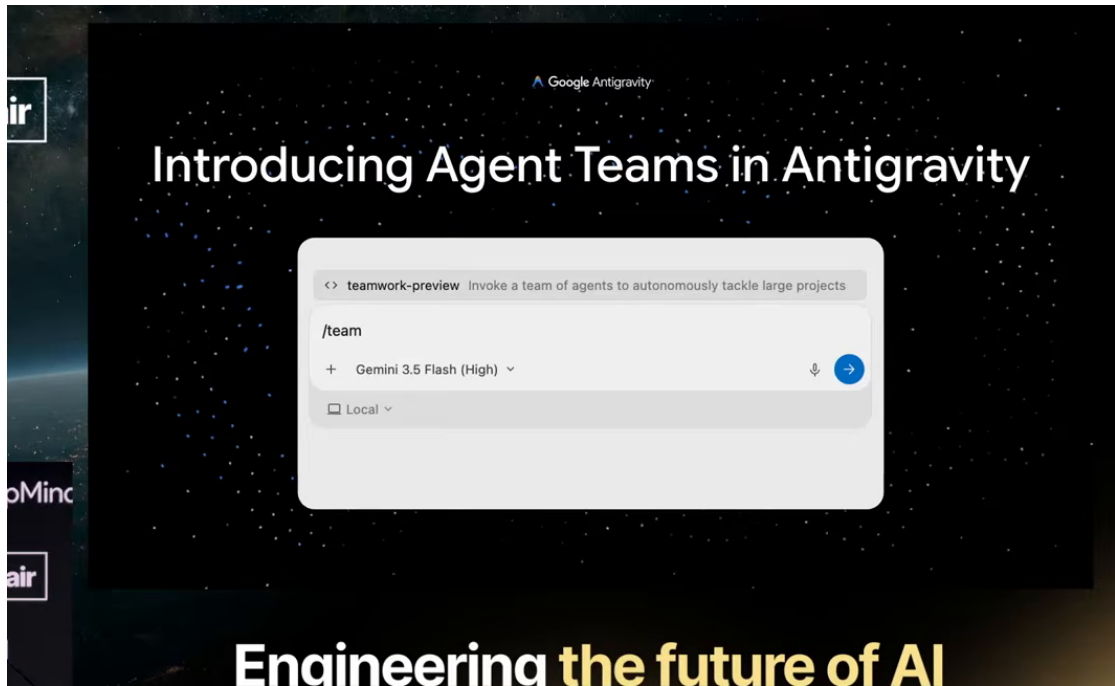


图 2: Agent Teams 的 ‘/team’ 入口与 teamwork-preview 模式。¹²

随后，主导代理（lead agent）接收任务澄清结果，管理一个可伸缩的团队。讲者用 Avengers 比喻角色组合：前端工程、后端工程、基础设施、QA 与设计等角色可被纳入同一任务。这里的关键不在角色名本身，而在角色随任务需求产生，并且任务成果需要重新汇总给用户。¹³

动

态子代理（dynamic sub-agents）指主导代理在运行时配置、提示并生成的专门智能体。它们可以并行执行，主导代理还可以为子代理选择与自身不同的模型。动态性强调运行时任务图，不能误解为预先固定的 worker 池。^a

^a视频原始时间：04:40:21–04:40:38。

从教学角度，这个过程可以压缩为四步：

1. 用户给出可澄清的目标与约束；
2. 主导代理追问缺失信息并形成任务边界；
3. 主导代理生成专业角色、选择模型并安排并行工作；
4. 汇总结果回到用户，由用户进一步审阅、追问或决定执行。

演讲没有公开任务图算法、模型路由策略、冲突解决方法或汇总评测标准。上述四步是对讲者描述的教学重组，不能被读作产品内部实现规范。

2.3 为何“更多智能体”不等于“更多可靠性”

讲者强调每个子代理可以独立运行，并以不同专业角色处理任务。独立性有助于并行覆盖不同工作面，却也引出工程约束：角色之间可能重复劳动，结论可能冲突，模型选择会改变成本与输出质量，

¹³视频原始时间：04:40:01–04:40:28。

外部工具权限可能被多个执行单元放大。团队规模需要与任务边界、预算、环境隔离与汇总验证共同设计。

“任意规模”描述的是讲者给出的产品能力方向。实际任务仍受上下文窗口、调用预算、资源配额、权限范围与验收机制约束。多智能体协调的第一项交付物应当是可审阅的任务边界与结果证据，而非数量本身。

2.4 规模案例：从应用原型到 OS kernel

讲者列举了系统实现的项目：能够在浏览器中直接编辑 RAW 照片的 photo editor，以及一个 messaging app；他称这类项目使用了数百个子代理，运行时间接近半天。¹⁴ 为了进一步压测团队能力，团队还演示了从零构建完整 OS kernel 并运行 Doom 的 hero run。¹⁵

该 OS kernel 演示的讲者报告数据为：93 个子代理、12 小时、15,000 次请求、20 亿 token，成本低于 1,000 美元。¹⁶ 这些数字适合用来理解多智能体任务的资源维度：并发角色数、持续时间、请求量、token 消耗与货币成本彼此关联。它们只描述一次展示性运行，未给出可复现条件、质量门槛、成功率、正常生产成本或安全评估，因此不能外推为日常开发承诺。

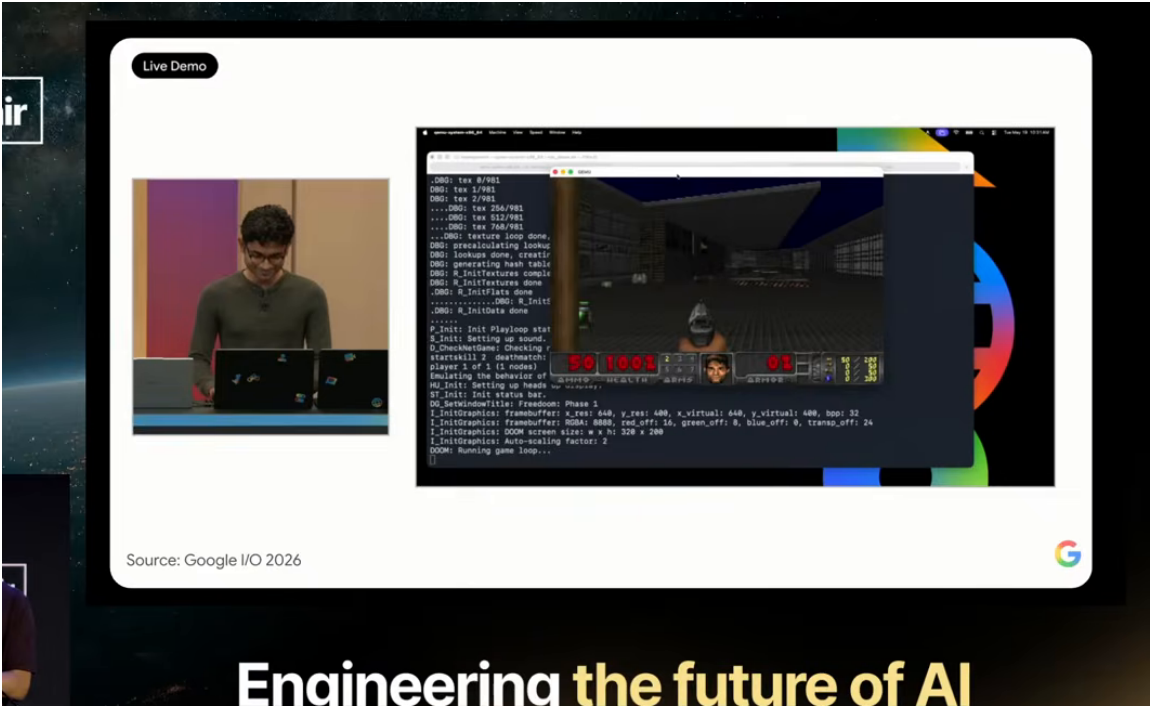


图 3: 讲者展示的 OS kernel 与 Doom 演示画面。¹⁷

这个案例的真正教学点是任务粒度变化。构建 kernel 并运行 Doom 涉及代码、构建、调试与验证等相互依赖的环节；它适合作为“可被拆分并需要汇总”的复杂任务例子。它并不证明任何复杂项目都应立即用大量智能体处理。

¹⁴ 视频原始时间：04:41:02–04:41:26。
¹⁵ 视频原始时间：04:41:24–04:41:56。
¹⁶ 视频原始时间：04:42:02–04:42:16。

2.5 本章小结

Agent Teams 将用户任务转成澄清、角色生成、并行执行和结果汇总的协作过程。主导代理负责协调，动态子代理承担专门工作。OS kernel/Doom 运行展示了这种结构可以触及的规模，同时也暴露了成本、验证、权限与协调质量必须一起管理的事实。

3 研究评测自动化：任务图、事件与交互审阅

本章的结论是：讲者以并排评测为例，展示了动态任务图负责调查假设、持续事件入口负责把系统延伸到事件驱动运行、按需交互视图负责让人审阅和筛选发现的闭环。该闭环的评价标准不是子代理数量或自动化百分比，而是每个结论是否保留可检查的证据、权限边界和人工介入点。

3.1 一个可迭代的评测问题

讲者用 side-by-side eval analysis 说明内部研究 workflow。研究者面对同一组任务的多个 rollout，需要比较 control 与 experiment 两组结果，识别差异，解释差异原因，并据此设计下一轮实验。¹⁸ 此处 rollout 指一次任务设置下产生或执行的试验结果，并非生产环境中的发布 rollout。

传统流程需要大量 Jupyter notebook 的手工作。讲者描述的新流程从自然语言问题开始：研究者询问某项评测，已经带有 skills、并理解大型 monorepo 的代理计算差异值 (delta)，再把分析扩展成关于差异原因的调查。¹⁹

3.2 从 delta 到可审阅的假设网络

讲者说系统会启动研究专长代理，提出约 100 个关于 delta 原因的假设；接着为每个假设生成子代理并行钻研，最后映射回一份给研究者审阅的响应。²⁰ 这段机制把“给出一个差异”扩展为“提出、调查并筛选解释”。

该流程的价值在于把调查对象显性化：每个假设都成为可分派工作项，研究者随后可以追问证据、比较解释并决定下一步实验。风险也同时上升：100 个假设并不自动代表 100 条可靠解释，假设来源、评审准则、数据访问范围和失败分支都需要记录。

生

成式界面 (generative UI) 指模型依据当前任务和数据状态即时生成、以内联方式呈现的交互界面。讲者在该评测例子中说明，结果会从报告继续扩展为可浏览、下拉选择、过滤、分段与切片的交互视图。它服务于审阅与探索，不能等同于一段静态 Markdown 输出。^a

^a视频原始时间：04:44:25–04:44:41。

¹⁸视频原始时间：04:43:01–04:43:31。

¹⁹视频原始时间：04:43:29–04:44:04。

²⁰视频原始时间：04:44:00–04:44:28。

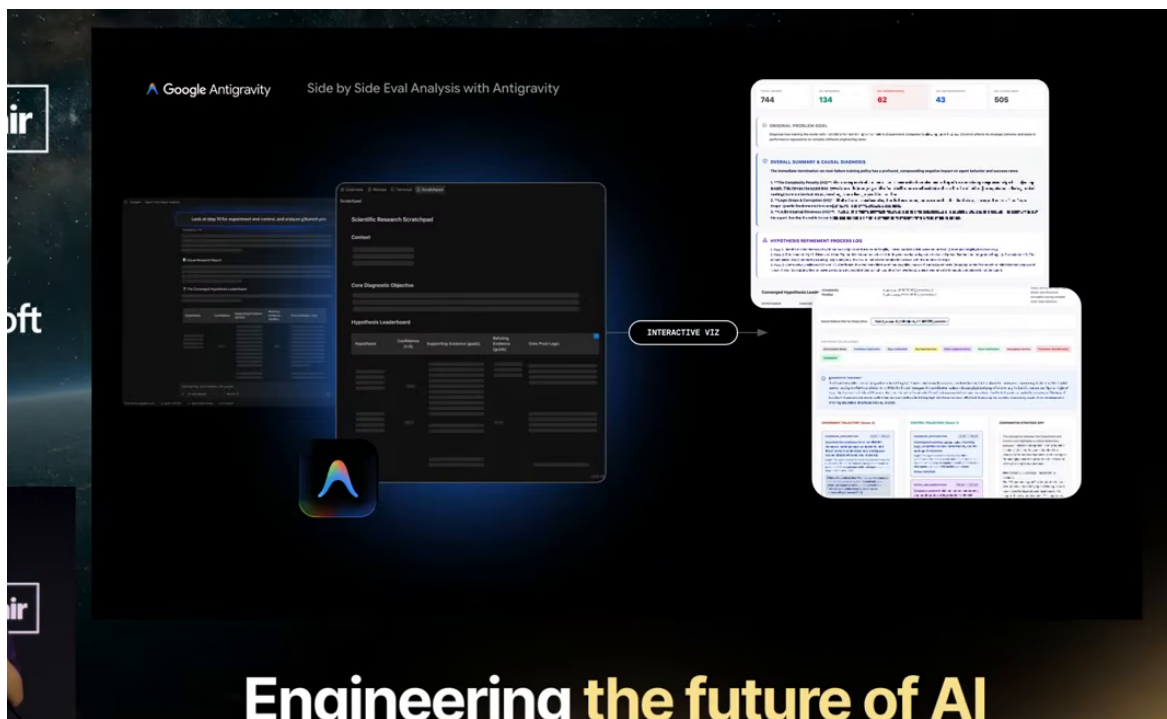


图 4: 讲者展示的 Side by Side Eval Analysis: 从输入分析面板经由 interactive visualization 到报告和可筛选结果。²¹

讲者称内部研究者仅用自然语言请求便自动化了约 90% 的工作流，原本的手工过程可缩短到分钟级。²² 这里必须保留“内部”与“讲者称”的范围：演讲未提供数据集、评价方法、失败率、访问权限或可复现基准。

3.3 Sidecar 伴随进程 (sidecar): 让系统获得持续的事件入口

评测例子中的调查从一次会话内的分析开始；若系统还要在新的数据、PR 或定时事件到来时继续工作，就需要一个持续的事件入口。为此，讲者提出 Sidecar 伴随进程。它是一类长生命周期的 plugin-style utility/process，可以监听外部世界，并设置自身触发器；列举的入口包括 SMS、webhooks、cron jobs 和 GitHub PRs。²³

S

idecar 伴随进程为智能体提供持续的事件入口。它与一次性工具调用的区别在于持续监听和触发：一次性调用围绕当前任务立即执行，Sidecar 则保留跨时间的连接与触发能力。讲者说 scheduled tasks/cron 的底层已使用这一原语。^a

^a视频原始时间：04:46:55–04:47:09。

这项原语把“什么时候开始工作”从会话内 prompt 扩展到外部事件。它也带来一组未被演讲展开的依赖：身份认证、密钥保管、事件真实性、幂等处理、触发频率、重试、审计记录和权限范围。讲者表示将在夏季发布该规范；这是一项未来计划陈述，本文不把它写作已发布的开放标准。²⁴

²²视频原始时间：04:43:41–04:44:58。

²³视频原始时间：04:46:24–04:46:58。

²⁴视频原始时间：04:47:08–04:47:14。

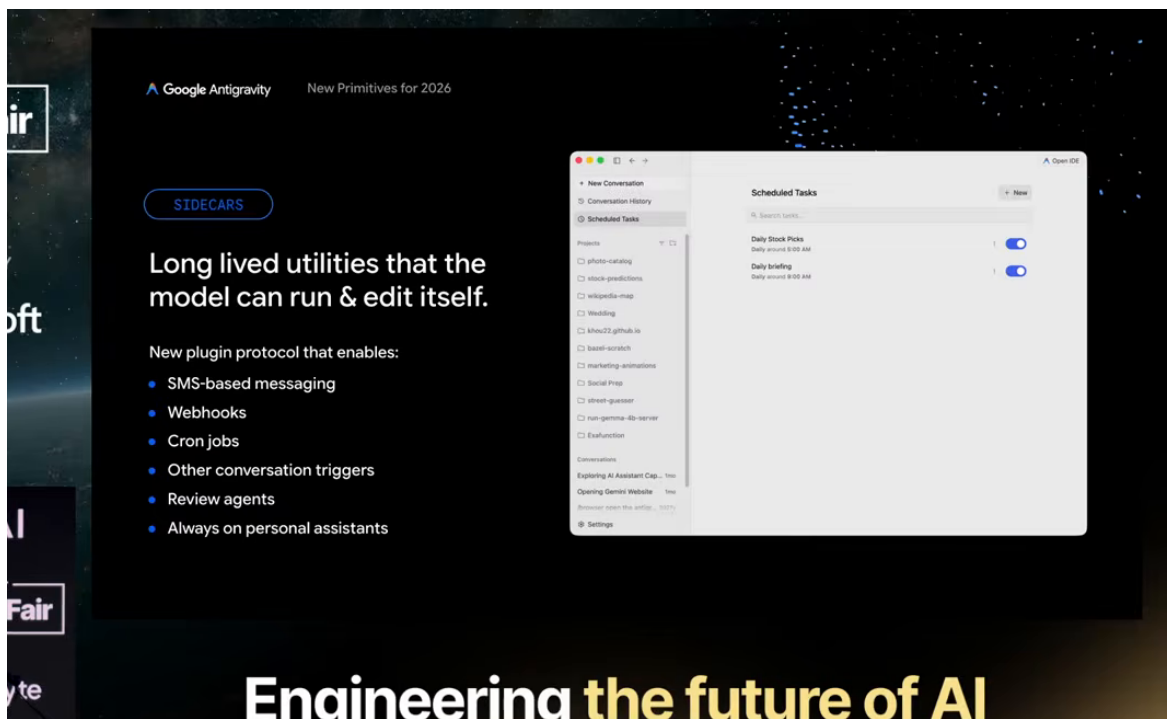


图 5: Sidecar 的长生命周期 utility 与 scheduled tasks 画面。²⁵

3.4 生成式界面：让结果按问题形状呈现

讲者认为 human-written specialized UIs 的固定形态将受到挑战，并声称 Anti-gravity 中 Gemini Flash 接近 900 tokens/s，约为许多 frontier model experiences 的 10 倍；他据此推断，用户的想法可以在数秒内转成嵌入会话的交互用例。²⁶ 性能数字和比较范围均来自讲者陈述，不能视为独立测试结论。

演讲给出的承载形式包括内联渲染的 Doom、柱状图、图表、表格，以及可进一步检查的交互内容。²⁷ 在评测 workflows 中，生成式界面把“系统说了什么”转成“研究者可以怎样查看、过滤和验证”。这正是它与普通报告的教学差异。

²⁶ 视频原始时间：04:47:18–04:47:49。

²⁷ 视频原始时间：04:47:46–04:48:08。

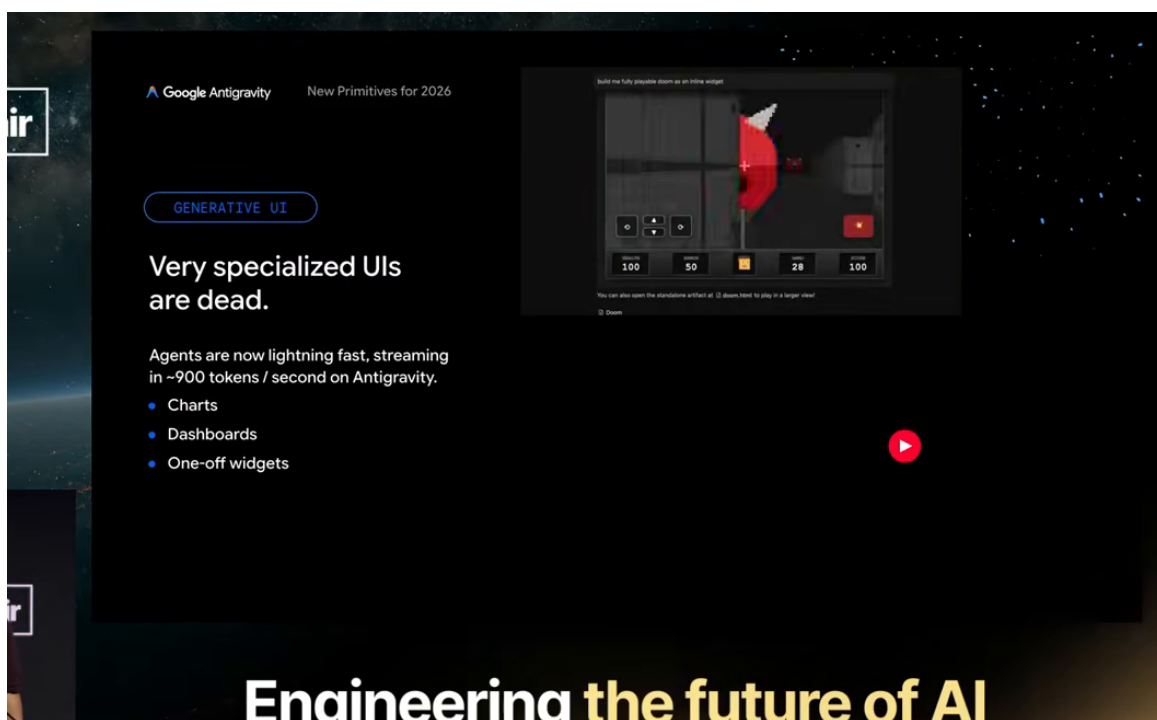


图 6: 讲者用固定键盘与专用界面对比说明按任务生成交互。²⁸

生

成式界面不自动保证显示状态正确、操作安全或结果可访问。涉及高风险动作时，界面仍需与权限、审计和人工确认协同。演讲没有给出这些机制的实现细节。

3.5 本章小结

评测案例把三个层面连接起来：动态任务图把假设调查并行化，Sidecar 把外部事件接入持续运行的系统，生成式界面把报告转为可操作的审阅面。该闭环的质量依赖于任务边界、数据与权限、假设评审和人类验证，不能只由自动化比例衡量。

4 总结与延伸：为下一代模型选择可扩展的原语

4.1 把三项原语放回同一张产品地图

演讲最后回收了三项 2026 原语：动态子代理、Sidecar 伴随进程与生成式界面。它们分别对应三个互补问题：复杂工作如何被专业化分配，系统如何从外部事件持续启动，人与系统如何围绕当前任务共同查看和操作结果。²⁹

这

三项原语可以形成一条教学上的工作链：主导代理生成动态子代理来处理任务图；Sidecar 把外部事件送入运行中的系统；生成式界面把汇总结果变成可审阅、可筛选的交互表征。该图是基于演讲概念的教学重组，不代表公开的内部架构图。

²⁹ 视频原始时间：04:48:43–04:48:55。

三者也不可以互相替代。子代理解决的是分工和执行拓扑；Sidecar 处理持续监听与触发；界面处理人类理解、检查和干预的表征。把它们混为“更多 agent 功能”会掩盖各自的失败模式：分工可能重复或冲突，触发可能不安全或不可控，界面可能误导或遗漏关键状态。

4.2 讲者的收束：产品应当给能力留出接口

讲者再次使用“let the model cook”的说法，并提出给 agent 产品的设计问题：产品中的哪些 primitives 会随着模型智能提升而扩展？³⁰ 他认为功能交付正在变得容易，真正困难之处转向选择能够在下一次模型变得更快、更好、更便宜时继续放大能力的功能与原语。³¹

这不是要求产品团队删除所有稳定界面或把每项决策交给模型。演讲中的终端权限经历恰好说明，能力释放需要边界与控制。更稳健的解释是：产品应把变化速度高、模型可承担、且可被验证的工作暴露为可伸缩原语，同时保留权限、审计、失败恢复和人工介入的明确位置。

4.3 给构建者的检查清单

对于一个准备引入多智能体能力的产品，可以用以下问题检查原语是否承担了清晰职责：

1. 任务能否被澄清、拆解、并行执行并重新汇总？每一步的输入、产物与验收责任由谁持有？
2. 哪些外部事件会触发工作？触发器是否具有身份、权限、频率、幂等和审计约束？
3. 用户需要哪些按需生成的状态、证据和交互，才能理解系统正在做什么，并在关键节点介入？
4. 模型提升后，当前抽象会放大能力，还是会因固定的角色、界面和流程成为瓶颈？

“

随智能扩展”可被视作一项产品组合检验。它要求团队同时看模型能力、工作流形状、风险边界与用户审阅面。只优化其中一项，通常无法形成可靠的智能体体验。

4.4 开放问题与谨慎推广

演讲没有说明 Agent Teams 的协调算法、冲突解决、模型路由、失败重试、长期运行成本或跨环境授权实现。Sidecar 的外部连接引出攻击面与运维问题；大量子代理引出预算、重复劳动与错误传播问题；生成式界面引出可信呈现、可访问性与安全操作问题。这些空白并不削弱演讲的产品启发，却限定了能够从演示直接推出的结论。

因此，最可迁移的结论不是“用更多智能体”，而是为已经变强的模型配置相称的任务分工、事件入口与审阅界面，并让每一层都保留可验证的边界。讲者的 kernel/Doom 与评测分析案例展示了规模化编排的潜力；实际产品仍需在可观测性、权限和质量门槛上获得独立证据。

4.5 本章小结

Anti-gravity 2.0 的演讲把产品设计的焦点放在可扩展原语上。动态子代理、Sidecar 与生成式界面构成一组分工、触发和交互的互补接口。模型能力持续提升时，能够持续释放价值的产品会同时具备任务边界、事件治理、结果审阅和人类控制。

³⁰ 视频原始时间：04:48:49–04:49:04。

³¹ 视频原始时间：04:49:02–04:49:22。